

Solution to Problem 1 in Problem Sheet - Week 4

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

```
In [1]: import numpy as np
import pandas as pd
from sklearn.cross_decomposition import CCA
```

Let us first load the data and inspect some of the rows of the data frame:

```
In [2]: cpus = pd.read_csv("cpus.csv")
cpus
```

```
Out[2]:
```

	name	syst	mmin	mmax	cach	chmin	chmax	perf	estperf
0	ADVISOR 32/60	125	256	6000	256	16	128	198	199
1	AMDAHL 470V/7	29	8000	32000	32	8	32	269	253
2	AMDAHL 470/7A	29	8000	32000	32	8	32	220	253
3	AMDAHL 470V/7B	29	8000	32000	32	8	32	172	253
4	AMDAHL 470V/7C	29	8000	16000	32	8	16	132	132
...	...	...	...	...	...	...	...	...	...
204	SPERRY 80/8	124	1000	8000	0	1	8	42	37
205	SPERRY 90/80 MODEL 3	98	1000	8000	32	2	8	46	50
206	STRATUS 32	125	2000	8000	0	2	14	52	41
207	WANG VS10	480	512	8000	32	0	0	67	47
208	WANG VS 90	480	1000	4000	0	0	0	45	25

209 rows × 9 columns

Select variables `perf` and `estperf` for data set `X`:

```
In [3]: X = cpus[["perf", "estperf"]]
X
```

```
Out[3]:
```

	perf	estperf
0	198	199
1	269	253
2	220	253
3	172	253
4	132	132
...	...	...
204	42	37
205	46	50
206	52	41
207	67	47
208	45	25

209 rows × 2 columns

And select the other variables except `name` for `Y`:

```
In [4]: Y = cpus[["syct", "mmin", "mmax", "cach", "chmin", "chmax"]]
Y
```

```
Out[4]:
```

	syct	mmin	mmax	cach	chmin	chmax
0	125	256	6000	256	16	128
1	29	8000	32000	32	8	32
2	29	8000	32000	32	8	32
3	29	8000	32000	32	8	32
4	29	8000	16000	32	8	16
...	...	...	...	...	...	...
204	124	1000	8000	0	1	8
205	98	1000	8000	32	2	8
206	125	2000	8000	0	2	14
207	480	512	8000	32	0	0
208	480	1000	4000	0	0	0

209 rows × 6 columns

Perform **canonical correlation analysis** using the class `CCA` from the module `sklearn.cross_decomposition`:

```
In [5]: cpus_cca = CCA()
X_c, Y_c = cpus_cca.fit_transform(X, Y)
```

The correlations of the first and second pairs of canonical variables are:

```
In [6]: print(np.corrcoef(X_c[:, 0], Y_c[:, 0])[0, 1])
print(np.corrcoef(X_c[:, 1], Y_c[:, 1])[0, 1])

0.9546400855741957
0.25071258533515084
```

The coefficients of the first pair for `X`:

```
In [7]: pd.Series(cpus_cca.x_weights_[0], index=X.columns)
```

```
Out[7]: perf      0.099932
estperf    0.994994
dtype: float64
```

Both positive for the two performance measures.

The coefficients of the first pair for `Y`:

```
In [8]: pd.Series(cpus_cca.y_weights_[0], index=Y.columns)
```

```
Out[8]: syct      0.162541
mmin      0.540968
mmax      0.738593
cach      0.199748
chmin     -0.011951
chmax      0.308824
dtype: float64
```

All positive except the one for `chmin` ("minimum number of channels").

Hence, the performance of a CPU, as described by the linear combination of the performance measures `perf` and `estperf`, is positively related to all apart from one of the quantitative characteristics of the CPU, which makes sense. The second pair is harder to interpret. The two performance measures have coefficients with opposite signs:

```
In [9]: pd.Series(cpus_cca.x_weights_[1], index=X.columns)
```

```
Out[9]: perf      0.994994
        estperf  -0.099932
        dtype: float64
```

One could possibly speculate that this represents some sort of discrepancy between the two performance measures. The signs of the loadings for `Y` are varied:

```
In [10]: pd.Series(cpus_cca.y_weights_[:,1], index=Y.columns)
```

```
Out[10]: syct      -0.239801
         mmin       0.379072
         mmax      -0.567257
         cach       0.427310
         chmin     -0.045112
         chmax       0.540731
         dtype: float64
```